



TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 1: The OS as a Resource Manager

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reîțman

CONTENTS

Abstract	3
Abbreviations	4
Introduction	5
1 File and directory management	6
1.1 Directory structure	6
1.2 Creation, copying, renaming, and removal	7
1.3 Ownership and permissions	7
2 Process management	9
2.1 Listing processes and identifying PID 1	9
2.2 Starting and terminating a background task	10
2.3 Inspecting process state through the kernel	11
3 Memory management	12
3.1 Physical memory and swap	12
3.2 Resident memory of a sleeping process	12
4 Device and storage management	14
4.1 Root filesystem and disk usage	14
4.2 Device entries and file-like interfaces	15
Conclusions	16

ABSTRACT

This laboratory examines how an operating system manages files, processes, memory, and devices using an Ubuntu virtual machine. Terminal commands were used to create and remove files, change access permissions, inspect running processes, terminate a background task, and read kernel information through the process filesystem. Memory and storage commands provided quantitative snapshots of the machine. The file owner and group were both `victor`; the process listing contained approximately 222 processes; memory inspection reported 3.3 GiB of total memory and no configured swap. The root filesystem was mounted from `/dev/sda2` using `ext4`. Screenshots document each resource-management task. Values from different captures are treated as separate observations because resource usage changes during execution.

Keywords: file permissions, processes, memory, devices, storage, linux

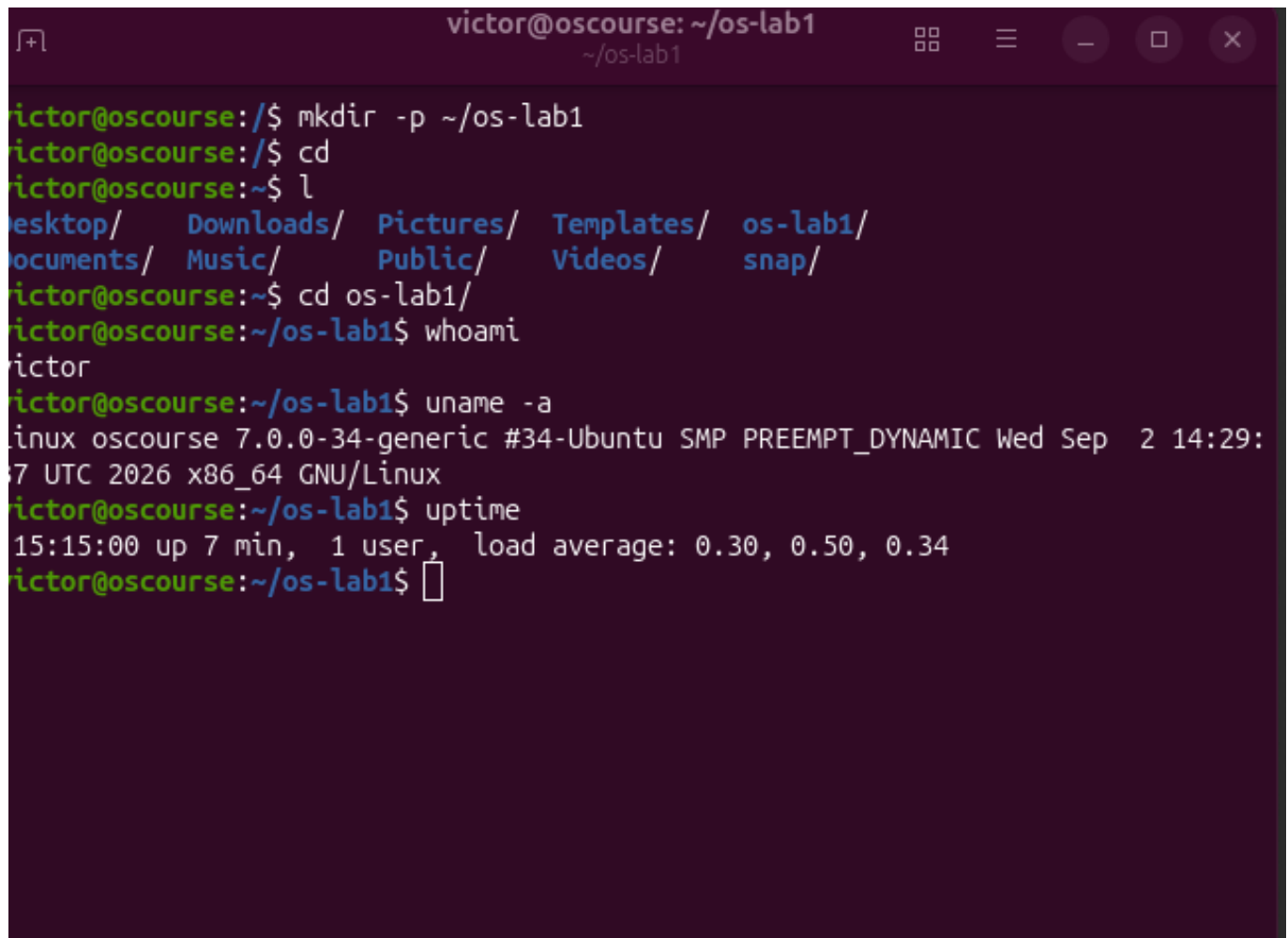
ABBREVIATIONS

OS	operating system
VM	virtual machine
PID	process identifier
RAM	random-access memory
RSS	resident set size
I/O	input/output
GiB, MiB, KiB	binary units: 2^{30} , 2^{20} , and 2^{10} bytes

INTRODUCTION

The operating system mediates between programs and hardware by allocating resources and enforcing access rules. This laboratory follows the four resource-management areas in the course handout: files, processes, memory, and devices [?]. The work was performed through the terminal in an Ubuntu VM. Figures throughout the report are the student's captured outputs.

The VM uses account `victor`, hostname `oscource`, and the `7.0.0-34-generic` Linux kernel on `x86-64`. The initial capture shows seven minutes of uptime. The work directory was `/home/victor/os-lab1`. The screenshot below records the environment. Later measurements were taken in separate sessions and should not be interpreted as simultaneous.



```
victor@oscource: ~/os-lab1
~/os-lab1

victor@oscource:/$ mkdir -p ~/os-lab1
victor@oscource:/$ cd
victor@oscource:~$ l
Desktop/   Downloads/ Pictures/  Templates/ os-lab1/
Documents/ Music/    Public/   Videos/   snap/
victor@oscource:~$ cd os-lab1/
victor@oscource:~/os-lab1$ whoami
victor
victor@oscource:~/os-lab1$ uname -a
Linux oscource 7.0.0-34-generic #34-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep  2 14:29:
37 UTC 2026 x86_64 GNU/Linux
victor@oscource:~/os-lab1$ uptime
15:15:00 up 7 min,  1 user,  load average: 0.30, 0.50, 0.34
victor@oscource:~/os-lab1$
```

1 FILE AND DIRECTORY MANAGEMENT

The filesystem presents data as a hierarchy of named objects. This part examined directory contents, basic file operations, ownership, and access permissions.

1.1 Directory structure

The home directory contains personal files and application settings. The `/etc` listing includes configuration entries such as `NetworkManager`, `X11`, and `adduser.conf`. Figure 1.1 shows these listings.

Two directories directly under the root are `/etc`, which contains system configuration, and `/home`, which contains users' home directories. The captured directory commands show the home and `/etc` contents; a direct listing of `/` was not captured.

```
victor@oscource:~/os-lab1$ ls -la
total 8
drwxrwxr-x  2 victor victor 4096 Sep 28 15:13 .
drwxr-x--- 16 victor victor 4096 Sep 28 15:13 ..
victor@oscource:~/os-lab1$ ls -la ~
total 80
drwxr-x--- 16 victor victor 4096 Sep 28 15:13 .
drwxr-xr-x  3 root  root  4096 Sep 28 14:33 ..
-rw-r----- 1 victor victor  108 Sep 28 15:12 .bash_history
-rw-r--r--  1 victor victor  220 Feb 13  2026 .bash_logout
-rw-r--r--  1 victor victor 3771 Feb 13  2026 .bashrc
drwx----- 13 victor victor 4096 Sep 28 15:01 .cache
drwxr-xr-x 15 victor victor 4096 Sep 28 14:59 .config
drwx-----  4 victor victor 4096 Sep 28 14:51 .local
-rw-r--r--  1 victor victor  807 Feb 13  2026 .profile
drwx-----  2 victor victor 4096 Sep 28 14:51 .ssh
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Desktop
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Documents
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Downloads
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Music
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Pictures
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Public
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Templates
drwxr-xr-x  2 victor victor 4096 Sep 28 14:51 Videos
drwxrwxr-x  2 victor victor 4096 Sep 28 15:13 os-lab1
drwx-----  5 victor victor 4096 Sep 28 15:00 snap
victor@oscource:~/os-lab1$ cd /etc
victor@oscource:/etc$ ls | head
ModemManager
NetworkManager
PackageKit
UPower
X11
adduser.conf
alsa
alternatives
anacrontab
apm
victor@oscource:/etc$
```

Figure 1.1 – Working, home, and configuration directory listings

1.2 Creation, copying, renaming, and removal

The command `echo` created `note.txt` containing `hello operating systems`. Copying it produced `copy.txt`, which was renamed to `renamed.txt`. Both files were 24 bytes long. After `rm renamed.txt`, the directory contained only `note.txt`. Figure 1.2 documents the commands and the before-and-after listings. The owner and group of both created files were `victor`.

```
victor@oscourse:/etc$ cd ~/os-lab1
victor@oscourse:~/os-lab1$ mkdir -p demo
victor@oscourse:~/os-lab1$ cd demo
victor@oscourse:~/os-lab1/demo$ echo "hello operating systems" > note.txt
victor@oscourse:~/os-lab1/demo$ cp note.txt copy.txt
victor@oscourse:~/os-lab1/demo$ mv copy.txt renamed.txt
victor@oscourse:~/os-lab1/demo$ ls -l
total 8
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 renamed.txt
victor@oscourse:~/os-lab1/demo$ cat note.txt
hello operating systems
victor@oscourse:~/os-lab1/demo$ rm renamed.txt
victor@oscourse:~/os-lab1/demo$ ls -l
total 4
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$
```

Figure 1.2 – Creating, copying, renaming, and deleting a file

1.3 Ownership and permissions

The initial mode was `-rw-rw-r--`. After `chmod 600 note.txt`, it became `-rw-----`. In this ten-character field, the first character identifies a regular file. The next three characters give the owner read and write permission without execute permission. The remaining groups of three give the group and other users no access. The command `chmod 644 note.txt` restored `-rw-r--r--`, allowing other users to read while retaining write permission only for the owner. Figure 1.3 shows all three modes.

```
victor@oscourse:~/os-lab1/demo$ ls -l
total 4
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$ ls -l note.txt
-rw-rw-r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$ chmod 600 note.txt
victor@oscourse:~/os-lab1/demo$ ls -l note.txt
-rw----- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$ chmod 644 note.txt
victor@oscourse:~/os-lab1/demo$ ls -l note.txt
-rw-r--r-- 1 victor victor 24 Sep 28 15:20 note.txt
victor@oscourse:~/os-lab1/demo$
```

Figure 1.3 – Permission changes from the initial mode to 600 and 644

2 PROCESS MANAGEMENT

A process is an executing program with an identifier and execution state. Process-listing tools and the kernel's `/proc` interface expose the tasks being managed by the OS.

2.1 Listing processes and identifying PID 1

The initial `ps aux | wc -l` command printed 223, including the output header. The header-free command `ps -e --no-headers | wc -l` printed 222. This indicates approximately 222 processes at that snapshot; counts can change even between consecutive commands.

Process 1 was `systemd`, running as `root` from `/usr/lib/systemd/systemd`. It is the initial userspace process and manages system services. Figure 2.1 records its identity and the process counts.

```
victor@oscourse:~/os-lab1/demo$ ps aux | head
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.5  0.4 25380 16684 ?        Ss   15:07   0:05 /usr/lib/systemd/systemd --switched-root --system --deserialize=51
root         2  0.0  0.0      0     0 ?        S    15:07   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    15:07   0:00 [pool_workqueue_release]
root         4  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-rcu_gp]
root         5  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-sync_wq]
root         6  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-kvfree_rcu_reclaim]
root         7  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-slub_flushwq]
root         8  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/R-netns]
root        10  0.0  0.0      0     0 ?        I<   15:07   0:00 [kworker/0:0H]
victor@oscourse:~/os-lab1/demo$ ps aux | wc -l
223
victor@oscourse:~/os-lab1/demo$ ps -e --no-headers | wc -l
222
victor@oscourse:~/os-lab1/demo$ ps -p 1 -o pid,comm,args
  PID COMMAND
    1 systemd
      /usr/lib/systemd/systemd --switched-root --system --deserialize=51
victor@oscourse:~/os-lab1/demo$
```

Figure 2.1 – Process listing, approximate count, and identity of PID 1

The live monitor in Figure 2.2 showed 251 tasks: one running and 250 sleeping, with no stopped or zombie tasks. This later snapshot differs from the earlier `ps` count because the VM's workload changed.

victor@oscourse: / — top											
top - 09:17:02 up 10 min, 1 user, load average: 0.29, 0.42, 0.35											
Tasks: 251 total, 1 running, 250 sleeping, 0 stopped, 0 zombie											
%Cpu(s): 22.0 us, 5.2 sy, 0.0 ni, 72.2 id, 0.3 wa, 0.0 hi, 0.4 si, 0.0 st											
MiB Mem : 3398.1 total, 202.1 free, 2147.7 used, 1311.4 buff/cache											
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1250.4 avail Mem											
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2849	victor	20	0	5107532	378188	117736	S	63.1	10.9	1:00.52	gnome-shell
8974	victor	20	0	812836	46396	30596	S	18.3	1.3	0:00.55	seahorse
8602	victor	20	0	4892420	275888	141760	S	12.0	7.9	0:01.76	gnome-character
8604	victor	20	0	590280	57088	39920	S	5.0	1.6	0:00.85	gnome-clocks
3851	victor	20	0	3789060	498120	167884	S	1.3	14.3	0:57.32	firefox
5516	victor	20	0	2774920	206452	92260	S	1.3	5.9	0:13.11	ptyxis
8598	victor	20	0	1118804	58188	47712	S	1.3	1.7	0:00.50	nautilus
40	root	20	0	0	0	0	I	1.0	0.0	0:00.27	kworker/u17:0-flush-8:0
74	root	20	0	0	0	0	S	1.0	0.0	0:01.44	kswapd0
2551	victor	20	0	10240	7720	4872	S	1.0	0.2	0:01.02	dbus-daemon
4753	victor	20	0	2835656	193776	87876	S	1.0	5.6	0:10.12	Isolated Web Co
3000	victor	20	0	388120	11504	7508	S	0.7	0.3	0:00.61	ibus-daemon
3393	victor	20	0	171904	7988	7304	S	0.7	0.2	0:00.15	ibus-engine-sim
3405	victor	20	0	3261664	50204	34584	S	0.7	1.4	0:01.23	gjs
8600	victor	20	0	591404	26612	23300	S	0.7	0.8	0:00.05	gnome-calculato
15	root	20	0	0	0	0	I	0.3	0.0	0:00.34	rcu_preempt
53	root	20	0	0	0	0	I	0.3	0.0	0:00.13	kworker/u17:2-flush-8:0
72	root	20	0	0	0	0	I	0.3	0.0	0:00.11	kworker/u20:1-events_unbound
4259	victor	20	0	2718860	144224	81712	S	0.3	4.1	0:03.57	Privileged Cont
4760	victor	20	0	2694104	135400	83232	S	0.3	3.9	0:03.61	Isolated Web Co
8596	victor	20	0	444608	36216	29620	S	0.3	1.0	0:00.14	gnome-control-c
1	root	20	0	26212	17048	11472	S	0.0	0.5	0:03.58	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	20	0	0	0	0	S	0.0	0.0	0:00.00	pool_workqueue_release
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-rcu_gp
5	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-sync_wq
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-kvfree_rcu_reclaim
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-slub_flushwq
8	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-netns
10	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H-kblockd
12	root	20	0	0	0	0	I	0.0	0.0	0:00.00	kworker/u16:0-ipv6_addrconf
13	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/R-mm_percpu_wq

Figure 2.2 – Live CPU, memory, and process information from top

2.2 Starting and terminating a background task

The command `sleep 300 &` started a background process with PID 9453. The shell's `jobs` command and the system-wide `ps` listing both showed it running. After `kill "$PID"`, the shell reported termination. The final process lookup printed only a header, indicating that the process was no longer present. Figure 2.3 records this lifecycle.

```

victor@oscourse:/$ sleep 300 & PID=$!
[1] 9453
victor@oscourse:/$ echo "Started PID: $PID"
Started PID: 9453
victor@oscourse:/$ jobs
[1]+  Running                  sleep 300 &
victor@oscourse:/$ ps -ef | grep '[s]leep'
victor      9453      5562  0 09:19 pts/0    00:00:00 sleep 300
victor@oscourse:/$ kill "$PID"
[1]+  Terminated              sleep 300
victor@oscourse:/$ wait "$PID"
victor@oscourse:/$ jobs
victor@oscourse:/$ ps -p "$PID" -o pid,comm,state
      PID COMMAND      S
victor@oscourse:/$ 

```

Figure 2.3 – Starting, finding, and terminating a background process

2.3 Inspecting process state through the kernel

A separate sleep process had PID 9765. Its directory under /proc contained entries such as cmdline, fd, maps, and status. The status file reported State: S (sleeping). This is interruptible sleep: the process waits rather than actively executing on the CPU. Figure 2.4 also shows the process and parent identifiers and its termination after inspection.

```

victor@oscourse:/$ sleep 300 & PID=$!
[1] 9765
victor@oscourse:/$ echo "Inspecting PID: $PID"
Inspecting PID: 9765
victor@oscourse:/$ ls /proc/"$PID"/
arch_status  cmdline      exe          ksm_stat    mem          numa_maps   personality  setgroups   status       uid_map
attr         comm         fd           latency     mountinfo    oom_adj     projid_map  smaps       syscall      wchan
autogroup   coredump_filter fdinfo       limits      mounts       oom_score   root        smaps_rollup task
auxv        cpu_resctrl_groups gid_map      loginuid    mountstats   oom_score_adj sched       stack       timers_offsets
cgroup      cwd          io           map_files   net          pagemap     schedstat   stat        timers
clear_refs  environ      ksm_merging_pages maps         ns           patch_state sessionid    statm       timerslack_ns
victor@oscourse:/$ head -20 /proc/"$PID"/status
Name: sleep
Umask: 0002
State: S (sleeping)
Tgid: 9765
Ngid: 0
Pid: 9765
PPid: 5562
TracerPid: 0
Uid: 1000 1000 1000 1000
Gid: 1000 1000 1000 1000
FDSize: 256
Groups: 4 24 27 30 46 100 111 114 1000
NSTgid: 9765
NSpid: 9765
NSpgid: 9765
NSSid: 5562
Kthreid: 0
VmPeak: 16112 kB
VmSize: 16112 kB
VmLck: 0 kB
victor@oscourse:/$ kill "$PID"
[1]+  Terminated              sleep 300
victor@oscourse:/$ 

```

Figure 2.4 – Kernel-exposed status of the sleeping process

3 MEMORY MANAGEMENT

The OS distributes physical memory between processes and uses reclaimable cache to improve performance. System totals and per-process memory fields provide different views of this allocation.

3.1 Physical memory and swap

Table 3.1 summarizes the `free -h` snapshot.

Table 3.1 – Observed memory totals

Field	Reported value
Total RAM	3.3 GiB
Used RAM	2.4 GiB
Free RAM	272 MiB
Available RAM	962 MiB
Configured swap	0 B

Free memory is unused memory, whereas available memory also accounts for memory that can be reclaimed for applications. Swap is backing space used to move eligible memory pages out of RAM; none was configured in this capture. The subsequent `/proc/meminfo` reading reported `MemTotal: 3479664 kB`. The commands were separate snapshots, so their free and available values need not match exactly.

3.2 Resident memory of a sleeping process

The process with PID 9978 reported `VmRSS: 7752 kB`, approximately 7.57 MiB. This is more than the source-level simplicity of `sleep` alone suggests, but an executing program still requires code, stack, runtime data, and possibly shared library pages. RSS counts resident shared pages as well as private pages and therefore does not measure exclusively owned memory. Figure 3.1 records both system and process readings.

```

victor@oscourse:/$ free -h
              total        used        free      shared  buff/cache   available
Mem:          3.3Gi        2.4Gi        272Mi        94Mi        977Mi        962Mi
Swap:          0B           0B           0B
victor@oscourse:/$ head -6 /proc/meminfo
MemTotal:        3479664 kB
MemFree:         289568 kB
MemAvailable:    995940 kB
Buffers:          7448 kB
Cached:          916392 kB
SwapCached:        0 kB
victor@oscourse:/$ sleep 300 & PID=$!
[1] 9978
victor@oscourse:/$ grep VmRSS /proc/"$PID"/status
VmRSS:           7752 kB
victor@oscourse:/$ kill "$PID"
[1]+  Terminated                  sleep 300
victor@oscourse:/$ 

```

Figure 3.1 – System memory, swap, and resident memory of sleep

4 DEVICE AND STORAGE MANAGEMENT

Storage tools connect logical mount points with the devices that provide their data. The device directory also exposes file-like interfaces for I/O.

4.1 Root filesystem and disk usage

The root filesystem `/` was mounted from `/dev/sda2`, using `ext4`, as confirmed by `findmnt /`. The `df -h` output reported 49 GiB total, 7.5 GiB used, 39 GiB available, and 17% utilization. The block-device listing showed a 50 GiB virtual disk named `sda`. The working folder occupied 12 KiB according to `du -sh`. Figure 4.1 connects the disk, partition, and mount point.

```
victor@oscourse:/$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           680M  2.7M  677M   1% /run
/dev/sda2       49G   7.5G   39G  17% /
tmpfs           1.7G   0    1.7G   0% /dev/shm
none            1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.7G   7.4M   1.7G   1% /tmp
none            1.0M   0    1.0M   0% /run/credentials/systemd-resolved.service
tmpfs           340M   92K   340M   1% /run/user/1000
/dev/sr0        51M   51M    0 100% /run/media/victor/VBox_GAs_7.2.18

victor@oscourse:/$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0       7:0      0    4K  1 loop /snap/bare/5
loop1       7:1      0  66.8M  1 loop /snap/core24/1643
loop2       7:2      0    20M  1 loop /snap/desktop-security-center/151
loop3       7:3      0  260.3M  1 loop /snap/firefox/8763
loop4       7:4      0   16.5M  1 loop /snap/firmware-updater/226
loop5       7:5      0  614.5M  1 loop /snap/gnome-46-2404/164
loop6       7:6      0   91.7M  1 loop /snap/gtk-common-themes/1535
loop7       7:7      0    1.5M  1 loop /snap/hwctl/123
loop8       7:8      0   402M  1 loop /snap/mesa-2404/1839
loop9       7:9      0   18.8M  1 loop /snap/prompting-client/222
loop10      7:10     0   11.8M  1 loop /snap/snap-store/1390
loop11      7:11     0   50.1M  1 loop /snap/snapd/27710
loop12      7:12     0    828K  1 loop /snap/snapd-desktop-integration/391
loop13      7:13     0   14.2M  1 loop /snap/desktop-security-center/188
loop14      7:14     0   18.4M  1 loop /snap/prompting-client/228
sda         8:0      0   50G   0 disk
├─sda1      8:1      0    1M   0 part
└─sda2      8:2      0   50G   0 part /
sr0        11:0     1  50.7M  1 rom  /run/media/victor/VBox_GAs_7.2.18

victor@oscourse:/$ du -sh ~/os-lab1
12K    ~/os-lab1

victor@oscourse:/$ findmnt /
TARGET SOURCE      FSTYPE OPTIONS
/       /dev/sda2 ext4    rw,relatime

victor@oscourse:/$
```

Figure 4.1 – Mounted filesystems, block devices, and working-folder size

4.2 Device entries and file-like interfaces

The entry `/dev/cdrom` points to `sr0`, the VM’s virtual optical drive. The storage listing shows VirtualBox Guest Additions media mounted from `/dev/sr0`. The entry `/dev/null` is a character device that discards written data and returns end-of-file when read.

The UNIX expression “everything is a file” describes how many resources, including devices, are exposed through operations such as opening, reading, and writing. It does not mean every resource is a regular disk file. Figure 4.2 shows device types, symbolic links, and mounts.

```
victor@oscource:/$ ls -l /dev | head
total 0
crw-r--r--  1 root  root    10, 235 Oct  1 09:06 autofs
drwxr-xr-x  2 root  root    420 Oct  1 09:11 block
drwxr-xr-x  2 root  root     80 Oct  1 09:06 bsg
crw-----  1 root  root    10, 234 Oct  1 09:06 btrfs-control
drwxr-xr-x  3 root  root     60 Oct  1 09:06 bus
lrwxrwxrwx+ 1 root  root      3 Oct  1 09:06 cdrom -> sr0
drwxr-xr-x  2 root  root   3780 Oct  1 09:07 char
crw-----  1 root  tty      5,   1 Oct  1 09:06 console
lrwxrwxrwx  1 root  root     11 Oct  1 09:06 core -> /proc/kcore
victor@oscource:/$ ls -l /dev/null
crw-rw-rw-  1 root root 1, 3 Oct  1 09:06 /dev/null
victor@oscource:/$ mount | head
tmpfs on /run type tmpfs (rw,nosuid,nodev,size=695936k,nr_inodes=819200,mode=755,inode64)
/dev/sda2 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,nosuid,size=779156k,nr_inodes=194789,mode=755,inode64)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,usrquota)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot,memory_hugetlb_accounting)
none on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
bpf on /sys/fs/bpf type bpf (rw,nosuid,nodev,noexec,relatime,mode=700)
victor@oscource:/$
```

Figure 4.2 – Device entries and filesystem mount interfaces

CONCLUSIONS

The OS manages files and directories, whose ownership and permissions were inspected with `ls -l`. It manages processes, visible with `ps`, and memory, visible with `free -h`. It also manages devices and storage, which were inspected with `lsblk`.

The observations demonstrate permission enforcement, process termination, kernel-exposed process state, and the association between a mounted filesystem and its device. Process counts and memory use varied between captures, as expected for a running system. All reported measurements are taken from the supplied terminal screenshots.